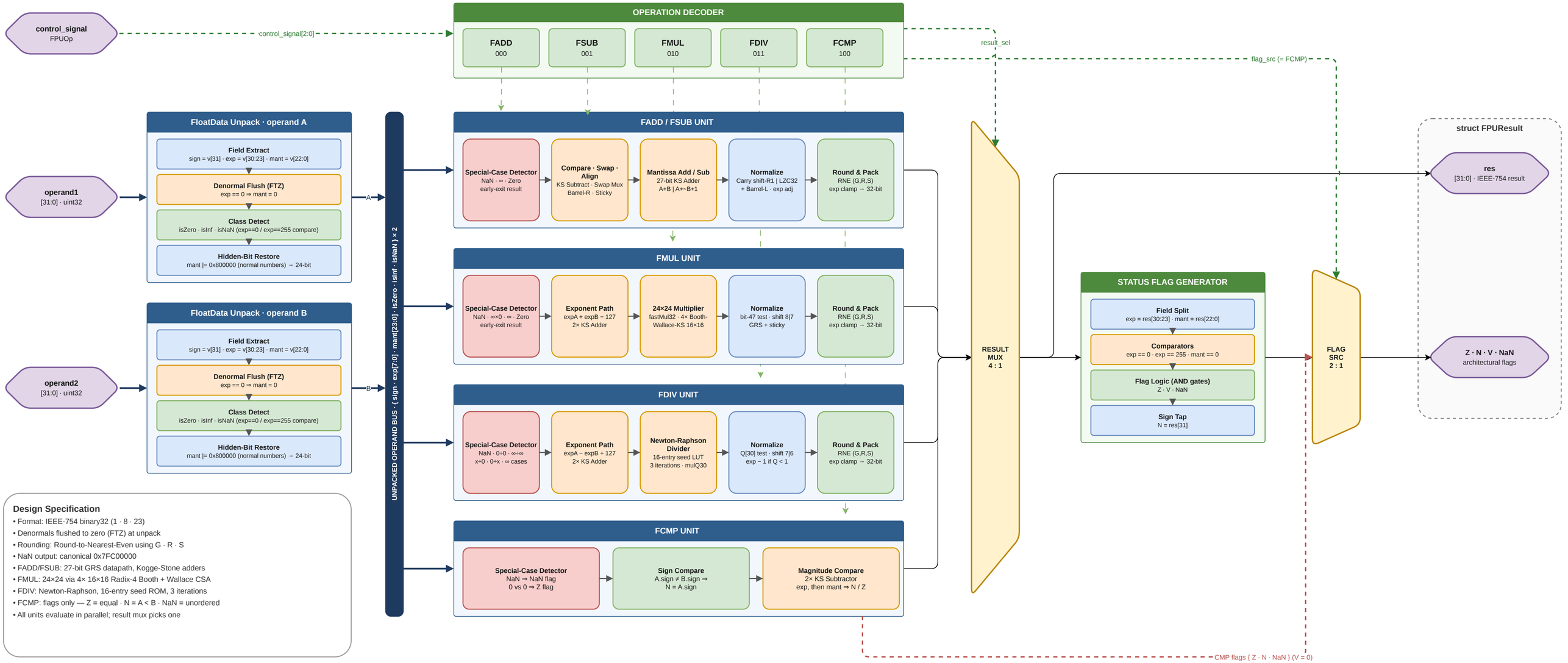
IEEE-754 FLOATING-POINT UNIT — TOP-LEVEL HARDWARE ARCHITECTURE



Design Specification

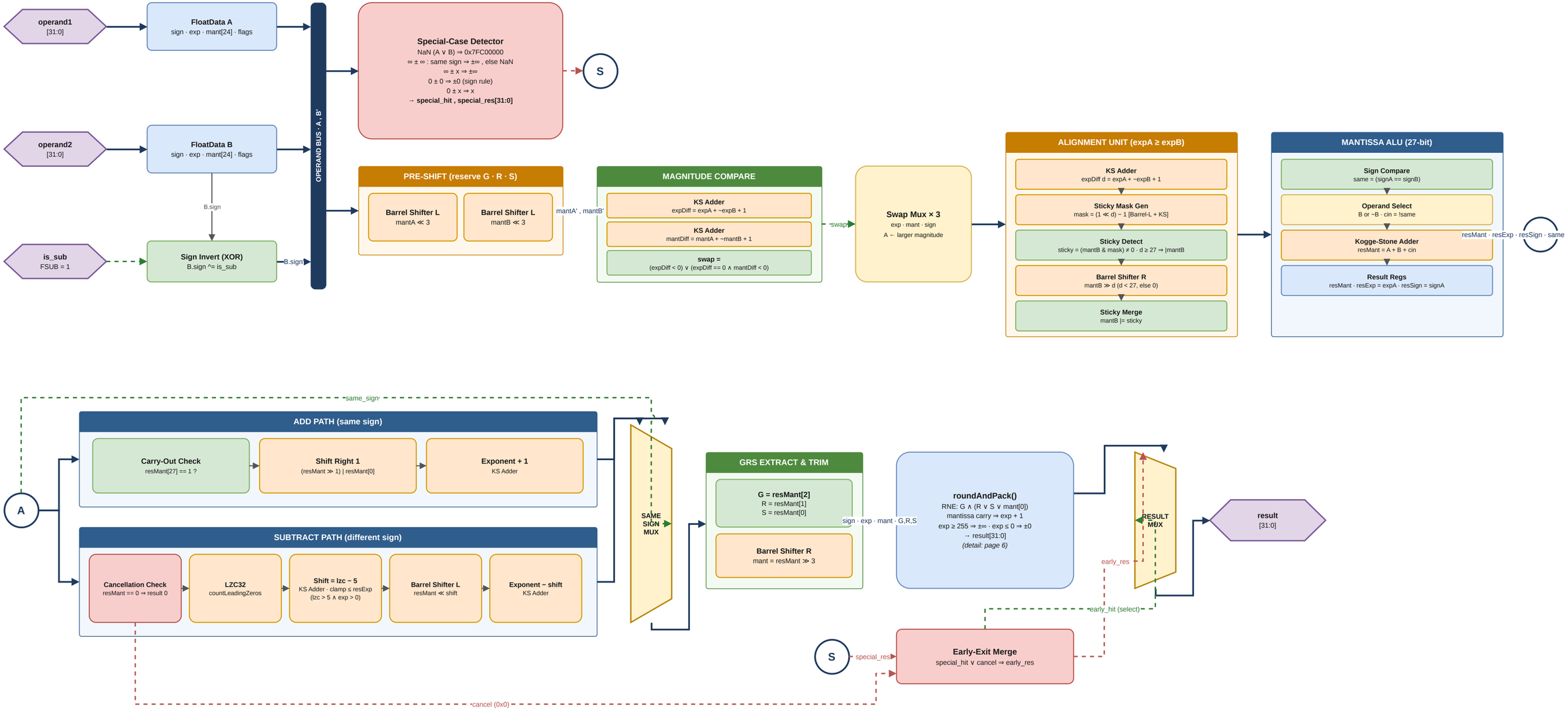
- Format: IEEE-754 binary32 (1 · 8 · 23)
- Denormals flushed to zero (FTZ) at unpack
- Rounding: Round-to-Nearest-Even using G · R · S
- NaN output: canonical 0x7FC00000
- FADD/FSUB: 27-bit GRS datapath, Kogge-Stone adders
- FMUL: 24×24 via 4× 16×16 Radix-4 Booth + Wallace CSA
- FDIV: Newton-Raphson, 16-entry seed ROM, 3 iterations
- FCMP: flags only — Z = equal · N = A < B · NaN = unordered
- All units evaluate in parallel; result mux picks one

LEGEND

Functional stage · Arithmetic primitive · Control / decision
Special-case logic · I/O port · Multiplexer
32-bit data path · Control signal

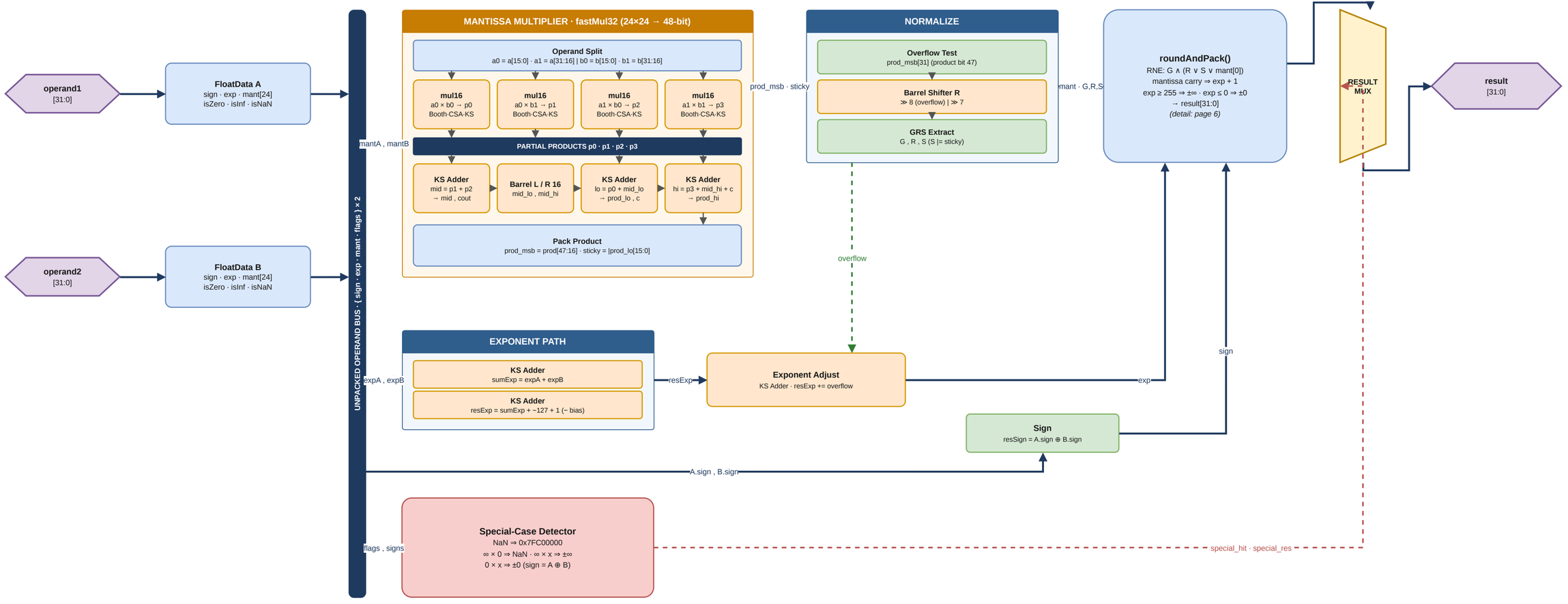
FADD / FSUB UNIT — executeAddSub() DATAPATH

Sign-invert for FSUB · special-case detect · GRS pre-shift · compare & swap · align + sticky · 27-bit Kogge-Stone add/sub · normalize · round & pack



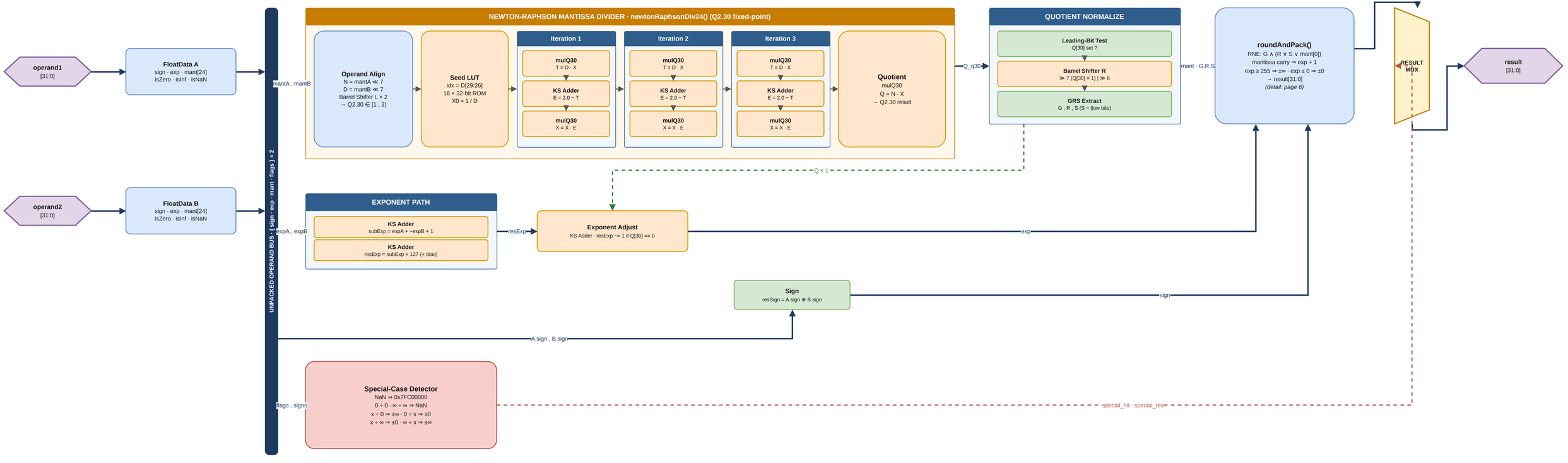
FMUL UNIT — executeMul() DATAPATH

24×24 mantissa multiply via four 16×16 Radix-4 Booth · Wallace-tree · Kogge-Stone multipliers · exponent add − bias · normalize · round & pack



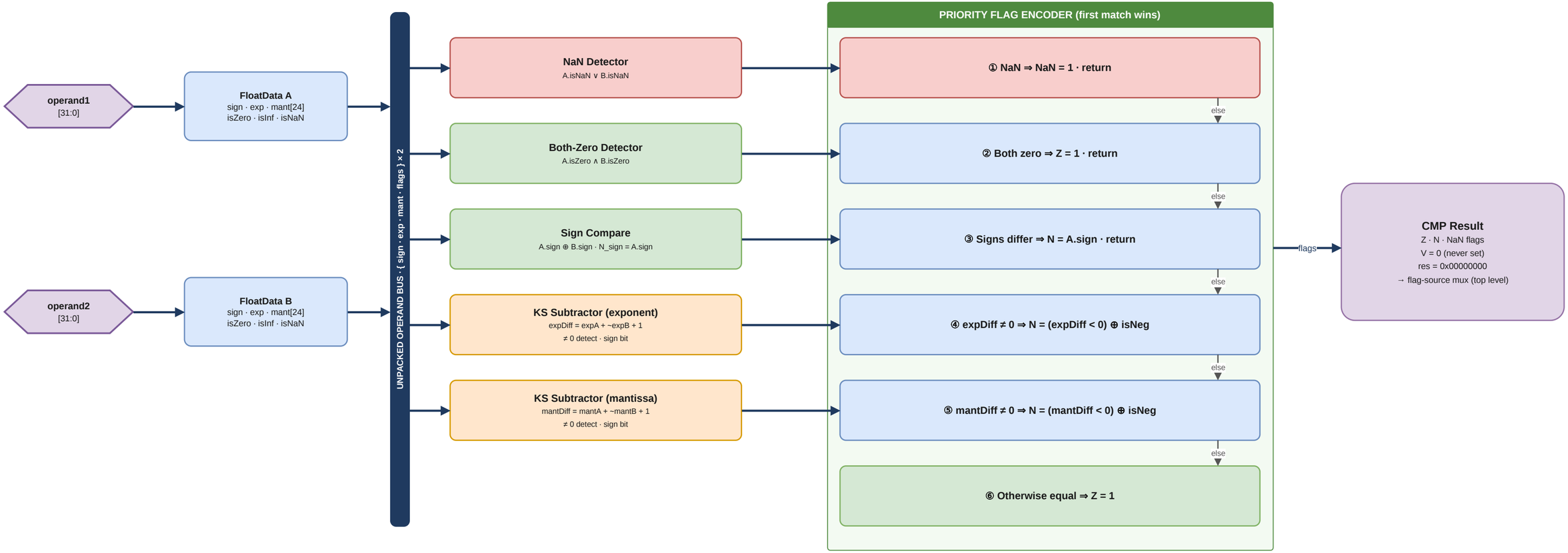
FDIV UNIT — executeDiv() DATAPATH

Newton-Raphson reciprocal (16-entry seed ROM + 3 iterations in Q2.30 fixed point) · quotient = N × (1/D) · exponent subtract + bias · normalize · round & pack



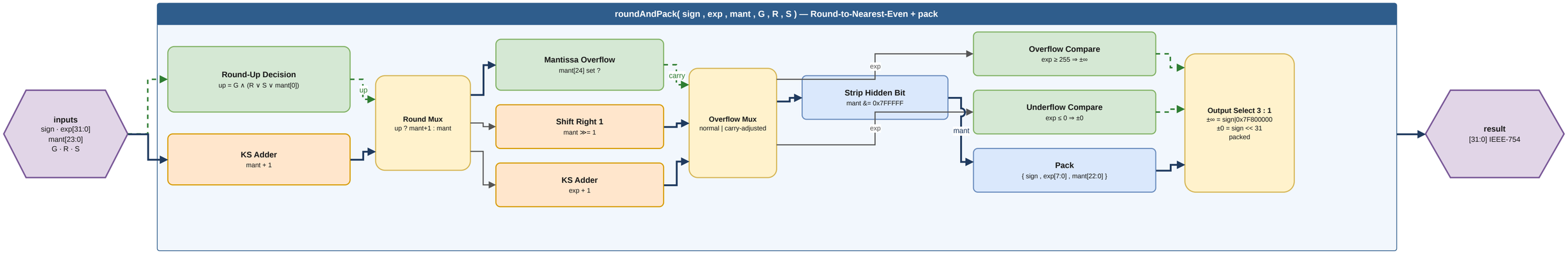
FCMP UNIT — executeCmp() DATAPATH

Parallel comparators feed a priority flag encoder (first match wins) · result is flags only: Z = equal · N = A < B · NaN = unordered



SHARED BLOCKS — roundAndPack() AND ARITHMETIC PRIMITIVE LIBRARY

roundAndPack is used by FADD/FSUB, FMUL and FDIV · primitives are black-box building blocks reused across the FPU



SHARED ARITHMETIC PRIMITIVE LIBRARY (black-box building blocks)

koggeStoneAdder32
a[31:0] , b[31:0] , cin → sum[31:0] , cout
Parallel-prefix · 5 stages (1,2,4,8,16)

barrelShiftLeft / Right
data[31:0] , shamt[4:0] → out[31:0]
5 mux stages (1,2,4,8,16) · shamt ≥ 32 ⇒ 0

carrySaveAdder (3:2)
a , b , c → sum , carry
sum = a⊕b⊕c · carry = maj(a,b,c) ≪ 1

countLeadingZeros32
x[31:0] → n[5:0]
Binary-search tree (16,8,4,2,1) · x = 0 ⇒ 32

FloatData Unpack
v[31:0] → sign , exp , mant[23:0]
isZero · isInf · isNaN · FTZ

boothSelect (Radix-4)
M , −M , window[2:0] , shift → pp[31:0]
{0 , ±M , ±2M} then Barrel-L

boothWallaceKoggeMul16
M[15:0] , Q[15:0] → P[31:0]
9 Booth PPs → CSA 9→6→4→3→2 → KS

fastMul32
a[31:0] , b[31:0] → { hi , lo }[63:0]
4× mul16 + 3× KS combine

mulQ30
a , b (Q2.30) → y (Q2.30)
fastMul32 → (hi ≪ 2) | (lo ≫ 30)

newtonRaphsonDiv24
N[23:0] , D[23:0] → Q (Q2.30)
Seed ROM + 3 NR iterations · N × (1/D)

Instantiation hierarchy (multiply / divide path)

newtonRaphsonDiv24 → **mulQ30** → **fastMul32** → **boothWallaceKoggeMul16**

boothSelect · carrySaveAdder · koggeStoneAdder32 · barrelShift